minimum size of 4 bytes would have given the answer as -2102132736 due to overflow.

```
#include<stdio.h>
int main( )
{
     long long int f=1;
     int a=20,i=1;
     for(i=1; i<=a; i++)
     {
        f = f * i;
     }
     printf("the factorial is %lld\n", f);
     return 0;
}
```

The program can be compiled by executing the command *gcc -std=c99 fact.c* or the command *gcc fact.c* on the terminal. The second command works because, by default, *gcc* compiler supports *long long int*. The output of the C99 program is shown in Figure 1.

Features like variable length arrays, better support for

```
File  Edit  View  Search  Terminal  Help
[deepu@localhost C99]$ gcc -std=c99 fact.c
[deepu@localhost C99]$ ./a.out
The factorial is 2432902008176640000
```

Figure 1: Factorial of 20 in C99

IEEE floating point standard, support for C++ style one line comments (//), macros with variable number of arguments, etc, were also added in C99. The official documentation of *gcc* has this to say: '*ISO C99. This standard is substantially completely supported*'. The legal term '*substantially complete*' is slightly confusing but I believe it means that the *gcc* compiler supports almost all the features proposed by the standards documentation of C99. As mentioned earlier, the option *-std=c99* of *gcc* will compile programs in the C99 standard.

C11

C11 is the current and latest standard of the C programming language and, as the name suggests, this standard was adopted in 2011. The formal document describing the C11 standard is called ISO/IEC 9899:2011. With C11, seven more keywords were added to the C programming language, thereby making the total number of keywords, 44. The seven keywords added to C99 are *_Alignas*, *_Alignof*, *_Atomic*, *_Generic*, *_Noreturn*, *_Static_assert* and *_Thread_local*. Consider the C11 program *noreturn.c* shown below, which uses the keyword *_Noreturn*.

```
#include<stdio.h>

_Noreturn fun( )
{
     printf("\nI TOO WILL BE PRINTED :) \n\n");
}

int main( )
{
     printf("\nI WILL BE PRINTED :) \n");
     fun( );
     printf("\nI WILL NOT BE PRINTED :( WHY? \n");
}
```

Figure 2 shows the output of the program *noreturn.c*. There are a few warnings because the *main( )* has one more line of code after the completion of the function *fun( )*. So why was the last *printf( )* in *main( )* not printed? This is due to the difference between a function returning *void* and a function with the *_Noreturn* attribute. The keyword *void* only means that the function does not return any values to the callee function, but when the called function terminates the program, the program counter register makes sure that the execution continues with the callee function. But in case of a function with the *_Noreturn* attribute, the whole program terminates after the completion of the called function. This is the reason why the statements after the function call of *fun( )* didn't get executed.

The function *gets( )* caused a lot of mayhem and so was

```
File  Edit  View  Search  Terminal  Help
[deepu@localhost C11]$ gcc -std=c11 noreturn.c
noreturn.c:3:11: warning: return type defaults to 'int' [-Wimplicit-int]
 _Noreturn fun()
           ^~~
noreturn.c: In function 'fun':
noreturn.c:6:1: warning: 'noreturn' function does return
 }
 ^
[deepu@localhost C11]$ ./a.out

I WILL BE PRINTED :)

I TOO WILL BE PRINTED :)
```

Figure 2: *_Noreturn* keyword in C11

deprecated in C99, and completely removed in C11. Header files like *<stdnoreturn.h>* are also added in C11. Support for concurrent programming is another important feature added by the C11 standard. Anonymous structures and unions are also supported in C11. But unlike C99, where the implementation of variable length arrays was mandatory, in C11, this is an optional feature. So even C11-compatible compilers may not have this feature. The official documentation of *gcc* again says that '*ISO C11, the 2011 revision of the ISO C standard. This standard is substantially completely supported*'. So, let us safely assume that the *gcc* compiler supports most of the features proposed by the C11 standard documentation. The option *-std=c11* of *gcc* will compile programs in C11 standard. The C Standards Committee has no immediate plans to come up with a new standard for C. So, I believe we will be using C11 for some time.

Continued on page 74....